Lecture 4: Strings

INFO 1100: Introduction to Programming

Before we start...

Before we start…

Assignment extra credit

Before we start…

End of 1st week
survey

Before we start…

Remember to ask
ahead of time if you
need accommodations!

Before we start...

Remember our AI policy

Before we start...

Remember our AI policy

AI Policy

Some assignments will explicitly request that you use an AI chatbot. For all other work, you are expected to complete it without using any AI assistance. Failure to follow this policy will result in an F for the assignment and may result in additional consequences.

Before we start…

Remember our AI policy

Question 1

Consider what real-world data we might represent as strings in a program. List 2-3 examples.

Before we start...

Remember our AI policy

```
Consider what real-world data we might represent as strings in a
program. List 2-3 examples.
```

Here are a few examples of real-world data commonly represented as strings:

1. **Names** — A person's full name, like `"Jane Doe"`, is text with no numeric meaning, making it a natural fit for a string.
2. **Phone numbers** — Despite containing digits, phone numbers like `"(555) 867-5309"` include formatting characters and are never used in arithmetic, so they're stored as strings.
3. **Addresses** — A mailing address like `"42 Elm Street, Springfield, IL 62704"` combines words, numbers, and punctuation into a single piece of text.

Before we start...

Remember our AI policy

Consider what real-world data we might represent as strings in a program. List 2-3 examples.

Examples of real-world data commonly represented as strings include:

- **People's names**
  - Examples: `"Rebecca Hicke"`, `"John Smith"`
  - Stored as strings because they consist of text characters.
- **Addresses**
  - Examples: `"123 Main Street, Ithaca, NY"`, `"221B Baker Street"`
  - Addresses contain letters, numbers, spaces, and punctuation.
- **Contact information**
  - Examples: `"rebecca@example.com"` (email address), `"(607) 555-1234"` (phone number)
  - Even though phone numbers contain digits, they are typically stored as strings because they are identifiers rather than values used in calculations.

Other common examples include book titles, course codes, ZIP codes, usernames, and website URLs.

Strings

Strings

a type is a set of values + operations on those values

Strings

a type is a set of values + operations on those values

represent **text** data

Strings

Enter in the chat:

what data that ***you*** find interesting would be represented as a string?

Strings

a type is a set of values + operations on those values

strings are ***sequences of characters***

Strings

a type is a set of values + operations on those values

strings are ***sequences of characters***

characters are individual symbols

Strings

a type is a set of values + operations on those values

strings are ***sequences of characters***

characters are individual symbols

(e.g. letters, digits, punctuation, white space)

Strings

sequences of characters

written in single (‘)
or double (“) quotes

Strings

sequences of characters

written in single (‘)
or double (“) quotes

```
“machine” == ‘machine’
```

String Indexing

String Indexing

we can access

part of the sequence

String Indexing

we can access

part of the sequence

```
name = "Rebecca"

first = name[0]

first → "R"
```

String Indexing

we can access

part of the sequence

Give ***3 examples***:

When might we want to index a string?

String Indexing

indexing

indicates the character in the sequence to select

String Indexing

indexing

“h e l l o _ w o r l d !”

String Indexing

indexing

“	h	e	l	l	o	_	w	o	r	l	d	!	”
0	1	2	3	4	5	6	7	8	9	10	11		

String Indexing

indexing

```
g = "hello world!"
```

g[3] → "l"

g[5] → " "

g[8] → "r"

String Indexing

negative **indexing**

String Indexing

negative **indexing**

“h e l l o _ w o r l d !”

String Indexing

negative **indexing**

```
"h e l l o _ w o r l d !"
 ... -4 -3 -2 -1
```

String Indexing

negative **indexing**

```
g = "hello world!"
```

```
g[3] → "l" ← g[-9]
g[5] → " " ← g[-7]
g[8] → "r" ← g[-4]
```

String Indexing

negative **indexing**

When could negative indexing be useful?

String Indexing

```
g = “hello world!”
```

```
g[x]
```

String Indexing

```
g = "hello world!"
```

```
g[x]
```

anything with
an integer value

String Indexing

```
g = "hello world!"
```

```
g[x]
```

anything with
an integer value

e.g. $x = 3, 5, x + 1$

String Indexing

```
g = "hello world!"
```

```
g[x]
```

anything with
an integer value

e.g. $x = 3, 5, x + 1$

Any other type will cause a **`TypeError`**

String Slicing

String Slicing

what if we want to take longer subsections of a string?

String Slicing

what if we want to take longer subsections of a string?

Give ***3 examples***:

When might we want to slice a string?

String Slicing

what if we want to take longer subsections of a string?

`[n:m]`

String Slicing

what if we want to take longer subsections of a string?

`[n:m]`

subsets a string
from `n`,
up to `m`

String Slicing

```
g = “hello world!”

“h e l l o _ w o r l d !”
 0 1 2 3 4 5 6 7 8 9 10 11
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[1:4]
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[1:4] → "ell"
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[5:10]
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[5:10] → " worl"
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[:4]
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[:4] → "hell"
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[4:]
```

String Slicing

```
g = "hello world!"

"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11

g[4:] → "o world!"
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[4:4]
```

String Slicing

```
g = "hello world!"
```

"h	e	l	l	o	_	w	o	r	l	d	!"
0	1	2	3	4	5	6	7	8	9	10	11

```
g[4:4] → ""
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[-5:-1]
```

String Slicing

```
g = "hello world!"
```

"h	e	l	l	o	_	w	o	r	l	d	!"
0	1	2	3	4	5	6	7	8	9	10	11

```
g[-5:-1] → "orld"
```

String Slicing

```
g = “hello world!”
```

```
“h e l l o _ w o r l d !”
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[:]
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[:] → "hello world!"
```

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

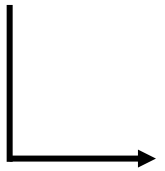
using `len()`

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

using `len()`



characters in a string

String Slicing

```
g = "hello world!"
```

```
"h e l l o _ w o r l d !"
 0 1 2 3 4 5 6 7 8 9 10 11
```

```
g[5:len(g)-3]
```

String Slicing

```
g = "hello world!"
```

"	h	e	l	l	o	_	w	o	r	l	d	!	"
	0	1	2	3	4	5	6	7	8	9	10	11	

```
g[5:len(g)-3] → " wor"
```

String Slicing

`[n:m]`

subsets a string
from `n`,
up to `m`

why do we exclude `m`?

String Slicing

why do we exclude **`m`**?

`[n:m]`

elegance over obviousness

String Slicing

why do we exclude **`m`**?

`[n:m]`

elegance over obviousness

`len(g[i:j])` is `j-i`

String Slicing

why do we exclude **`m`**?

`[n:m]`

elegance over obviousness

`len(g[i:j])` is `j-i`

`len(g[2:5])` is `5-2` is `3`

String Slicing

why do we exclude **`m`**?

`[n:m]`

elegance over obviousness

`len(g[i:j])` **is** `j-i`

`len(g[2:5])` is `5-2` is `3`

`g[0:i] + g[i:len(g)]`

splits the string in two

Strings are **immutable**

Strings are **immutable**

---

you **can't** change a string!

Strings are **immutable**

```
g[1] = "x"
```

will result in

```
TypeError: 'str' object does not
    support item assignment
```

Strings are **immutable**

Enter in the chat:

When might this matter?

String **methods**

String **methods**

use method syntax

String **methods**

function calls:

```
<function name>(<argument(s)>)
```

String **methods**

function calls:

`<function name>(<argument(s)>)`

e.g.

```
len('abc')
```

String **methods**

---

function calls:

```
<module name>.<function name>(<argument(s)>)
```

String **methods**

function calls:

<module name>.<function name>(<argument(s)>)

e.g.

```
math.sqrt(25)
```

String **methods**

method syntax:

```
<expression>.<method name>(<argument(s)>)
```

String **methods**

method syntax:

<expression>.<method name>(<argument(s)>)

e.g.

```
'abc'.index('c')
```

String **methods**

method syntax:

<expression>.<method name>(<argument(s)>)

e.g.

```
'abc'.index('c')
```

```
g.upper()
```

String **methods**

`index()`

String **methods**

```
s.index(c)
```

String **methods**

`s.index(c)`

finds the starting
index of a substring
`c` in string `s`

String **methods**

`s.index(c)`

e.g.

`"hello world!".index("w")` → `6`

`"abc".index("bc")` → `1`

String **methods**

```
s.index(c)
```

When is this useful?

Questions?

(… I'll wait for 3)

For tomorrow:

(06/26 @ 11:30am)

- Lab 4
- Quiz tomorrow!

For tomorrow:

(06/26 @ 11:30am)

- Lab 4
- Quiz tomorrow!

Monday:

(06/29 @ 9:00am)

- Assignment 1

Tuesday:

(06/30 @ 11:30am)

- Readings + GRQs